Collective learning

Hermes supplies memory, history, and skills. This project adds transaction checks, evidence, export approval, and a reviewed release path.

CONTENTS

1. Private and shared state
2. Local acceptance
3. Approval and release

PRIVATE AND SHARED STATE

Learning changes the Worker's context, not model weights.

PRIVATE MEMORY

`USER.md` and `MEMORY.md` retain compact preferences and durable working facts.

PRIVATE SKILLS

`skills/private/` stores project-, customer-, or assignment-specific playbooks.

ROLE SKILLS

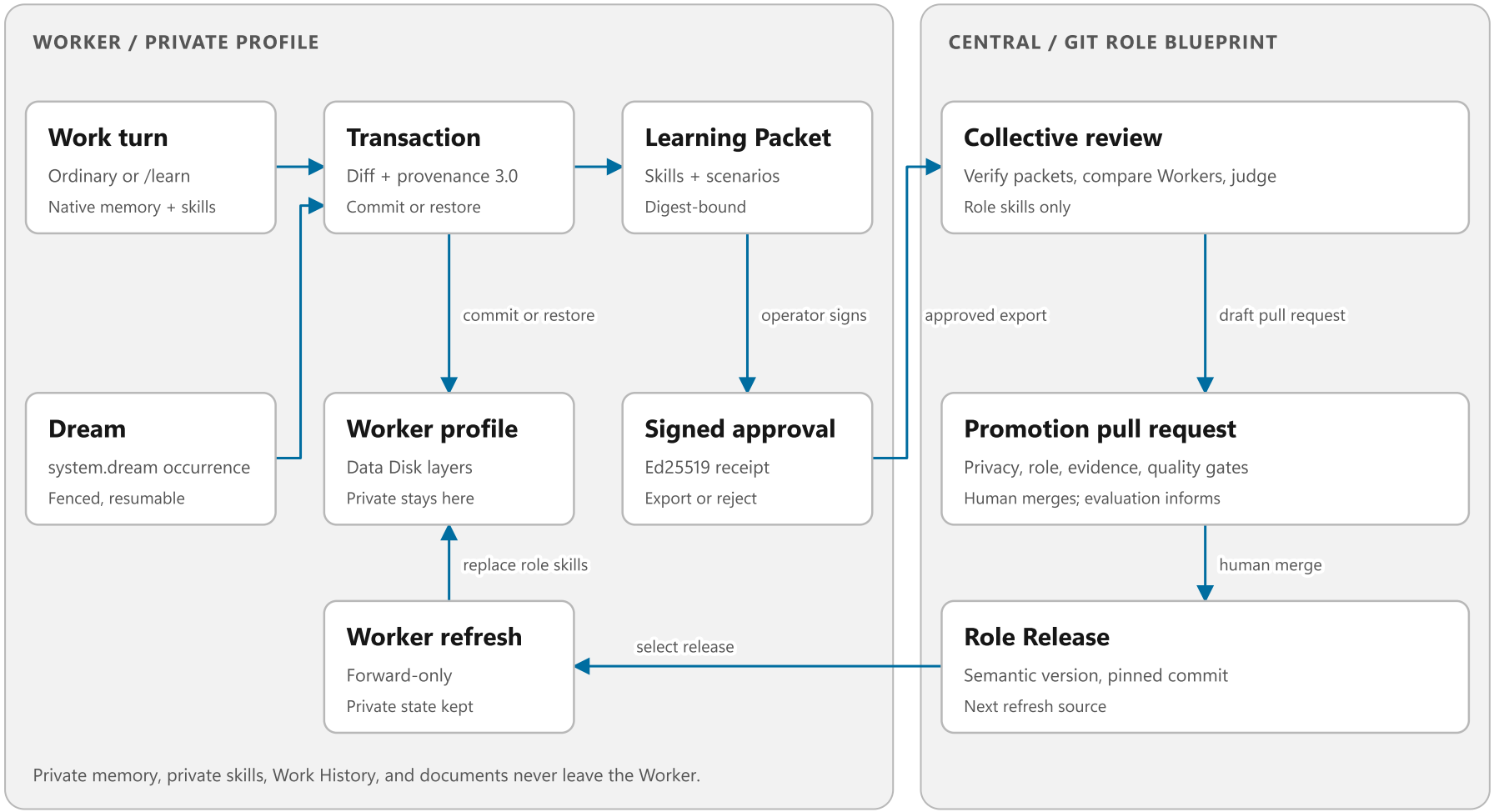
`skills/role/` starts from a reviewed Role Release and can receive governed local patches.

CANDIDATES

`skills/candidates/` holds generalized capabilities proposed locally for later review.

Raw Work History stays private. Only eligible role/candidate changes enter a Learning Packet. Generalization means removing private facts, not just moving the same text into another directory.

01 / Learning lifecycle



Local acceptance and shared promotion are separate decisions. Nothing crosses into the central zone without a signed approval.

> SOURCE
>
> Classification policy · Enforced learning boundary

01

Load compact context; retrieve deeper history when needed.

Native Hermes supplies the mechanisms. The Role Blueprint supplies their policy.

- **Fresh session:** active `SOUL.md` and compact memory provide starting context.
- **Relevant task:** skill names and descriptions allow discovery; full instructions load when selected.
- **Recall or Dream:** session search retrieves relevant Work History instead of injecting the entire database.

Memory files and private playbooks are Worker-wide. Session keys identify the caller, but native group transcripts are shared and rotate hourly by default. A user-specific key does not create a separate memory store.

> SOURCE
>
> Native runtime setup · Profile layout · Conversation boundary

LOCAL ACCEPTANCE

02

A turn can learn without approving its own export.

Ordinary work and /learn use the same transaction boundary.

Hermes uses native memory and skill tools. An explicit `/learn <instruction>` focuses one transactional turn on an adaptation. The model proposes the skill change and its provenance; the runtime independently derives what changed on disk.

A useful local improvement can remain active in this Worker before any shared release. That local acceptance neither exports private data nor grants permission to publish the skill to other Workers.

> SOURCE
>
> Turn routing and learning instructions · Turn reconciliation

03

Dream reflects across work, under the same rules.

A bounded reflection task, not a second autonomous runtime.

Review
Inspect outcomes, corrections, skills, and relevant history.

Classify
Keep private context private; generalize reusable behavior.

Reconcile
Accept valid changes or retain nothing.

The reviewed Dream Role Skill guides reflection. A system schedule invokes it through the gateway and Service Bus. Durable fenced phases recover completed responses; an unknown interrupted model outcome stops rather than triggering a blind replay. Ad-hoc execution does not consume the production occurrence.

> SOURCE
>
> Dream instructions · Durable coordinator

04

Trust the observed diff, not the model's narration.

The runtime owns hashes, journal records, and rejection.

1 **Snapshot under a lease**
Recover pending work and capture the governed/private skill state before the turn.

2 **Match change to provenance**
Derive the real diff, validate classification and privacy, and require matching evidence for each governed artifact.

3 **Commit or restore**
Accept the governed change set atomically with runtime-derived identity and hashes. Any rejection restores the entire governed snapshot and appends no records.

> Inspect the single-file contract

Provenance schema **3.0** permits `create` or `patch` on exactly one `SKILL.md` per artifact. `artifactPath` names the skill directory; `artifact.changedFiles` names that directory's one skill file. Scripts, references, auxiliary files, and deletion are outside the governed export lane.

The runtime, not the model, appends `learning/records.jsonl`. Attachment-bearing turns cannot turn document content into durable memory or skills.

SOURCE

Transaction implementation · Provenance schema

APPROVAL AND RELEASE

05

Package the improvement together with its proposed tests.

Learning Packet 2.0 carries content, cumulative evidence, and a verifiable identity.

Packet preparation reconstructs eligible changes from the Role Release baseline to the current skill hash. It includes each linked provenance record, the final skill file, Worker/release attribution, and the governed-state hash. The digest binds the packet's exact content.

Each agent-proposed scenario declares input, setup assumptions, scope, expected observable outcomes, and acceptance criteria. Criteria operate on `response.text` using `contains`, `not_contains`, or `equals`; they are data, not executable test scripts.

Privacy checks cover both skill text and test proposals. Raw memories, history, private skills, workspace files, credentials, and logs are excluded.

> SOURCE
>
> Packet reconstruction · Scenario contract

06

Approval binds the packet and the current Worker state.

A later edit makes the prior approval stale.

The runtime verifies an Ed25519-signed receipt against the packet digest, Worker, release commit, and governed-state hash. A signature over a different or stale packet cannot authorize export.

APPROVE EXPORT

Allows the exact reviewed packet to leave the Worker; it does not approve merging or release.

REJECT AND REFRESH

Records the signed rejection and permits discarding those governed local changes during the controlled refresh.

An unreviewed packet must not be treated as approved simply to unblock a refresh.

> SOURCE
>
> Receipt verification and refresh guard · Operator approval flow

07

Proposed tests express intent; independent cases test the claim.

Run baseline and candidate on the same cases and report the scope honestly.

Agent-written scenarios make the intended improvement inspectable. Reviewers still need independent regression cases: otherwise the author can describe a skill and the easiest test for that same description.

The evaluation script performs real model calls for both versions and records case outcomes and usage. Missing usage is an error, not zero cost. Results need the case set, model/settings, baseline, candidate, and packet binding to be interpretable.

> Inspect what this evaluator measures

The current runner injects selected skill text into the prompt and asserts response text. It does not measure native skill discovery, live MCP workflows, tool side effects, or general task success. Agent-proposed and independent cases must stay distinguishable; neither a signature nor a higher pass rate broadens the tested scope.

SOURCE

Evaluation runner · Evaluation contracts

08

Share through a Role Release, not by copying another Worker's profile.

Preserve private continuity while replacing governed behavior.

Review packet
Check privacy, evidence, skill quality, and role alignment.

Publish release
Pin the reviewed Role Blueprint to an immutable commit.

Refresh Worker
Resolve local governed changes, then install the selected release.

A forward-only refresh preserves private memories, private playbooks, and Work History. It replaces Role Skills and archives the prior release's candidates and provenance. It does not copy another Worker's private profile or silently merge unreviewed local edits.

> SOURCE
>
> Promotion review workflows · Refresh guard · Release installation

Learn locally, export only what a person approved, and share behavior through a reviewed Role Release.

▼ Work turn

Ordinary or /learn

Native memory and skill tools can change the profile during ordinary work. An explicit `/learn <instruction>` focuses one transactional turn on an adaptation. Attachment-bearing turns cannot turn document content into durable memory or skills.

▼ Dream

Scheduled reflection

A system schedule delivers `system.dream` through Service Bus and the gateway. The reviewed Dream Role Skill reviews, classifies, and reconciles. Durable fenced phases recover completed responses; an unknown interrupted outcome stops for inspection.

▼ Learning transaction

Runtime-owned boundary

The runtime snapshots governed and private skills under a lease, derives the real diff, and matches each governed artifact to provenance schema 3.0. It commits atomically or restores the whole snapshot and appends no records.

▼ Worker profile

Data Disk

`USER.md`, `MEMORY.md`, `skills/private/`, `skills/role/`, `skills/candidates/`, `learning/records.jsonl`, sessions, and schedules. Private layers never enter a Learning Packet.

▼ Learning Packet 2.0

Digest-bound export candidate

Reconstructs eligible role and candidate changes from the Role Release baseline, with provenance, final skill text, attribution, the governed-state hash, and agent-proposed scenarios. Privacy checks cover skill text and scenarios.

▼ Signed approval

Ed25519 receipt

The runtime verifies the signature against the packet digest, Worker, release commit, and governed-state hash. *Approve export* lets the exact packet leave; *reject and refresh* permits discarding those local changes.

▼ Collective review

Central workflow

Verifies packet digests and receipts, compares packets from several Workers, and judges which role skills generalize. Private skills and raw history are not inputs.

▼ Promotion pull request

Draft pull request with agentic gates

GitHub Agentic Workflows triage the change and review privacy, role alignment, evidence, and skill quality. Evaluation results inform the reviewer; a human merges.

▼ Role Release

Pinned Git commit

A semantic version pinned to an immutable commit of the Role Blueprint. It becomes the baseline for later Learning Packets.

▼ Worker refresh

Forward-only

Requires local governed changes to be exported or rejected first. Replaces role skills, archives prior candidates and provenance, and keeps private memory, private skills, and Work History.